

工具调用

如何在 Kimi API 中使用官方工具

📄 复制页面



Kimi 开放平台特别推出官方工具，您可以将 Kimi 官方工具**免费**集成到您自己的应用程序中，打造属于您的智能化商业产品！（目前 Kimi 开放平台官方工具执行限时免费，当工具负载达到容量上限时，可能会采取临时的限流措施）

本章节将为您详细介绍如何在您的应用中轻松调用和执行这些官方工具。

Kimi 官方工具列表

工具名称	工具描述
convert	单位转换工具，支持长度、质量、体积、温度、面积、时间、能量、压力、速度和货币的单位换算
web-search	实时信息及互联网检索工具。价格与可用性详情请见 联网搜索价格
rethink	智能整理想法工具
random-choice	随机选择工具
mew	随机产生猫的叫声和祝福的工具
memory	记忆存储和检索系统工具，支持对话历史、用户偏好等数据的持久化
excel	Excel 和 CSV 文件的分析工具
date	日期时间处理工具
base64	Base64 编码与解码工具
fetch	URL 内容提取 Markdown 格式化工具

🚀 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

quickjs

使用 Quick JS 引擎安全执行 JavaScript 代码的工具

KIMI
code_runner

开放平台

Python代码执行工具



>

调用 web_search 官方工具的示例

以下是一个 python 示例，以 web_search 官方工具为例，展示了如何通过 Kimi API 调用官方工具：

您也可以通过 Kimi 开发工作台来交互式体验 Kimi 模型和工具的能力，[前往开发工作台](#)

这里是您可以使用的 Kimi 官方 Formula 工具，您可以将 formula URI 增加到下方 demo 示例中体验：

moonshot/convert:latest , moonshot/web-search:latest ,

moonshot/rethink:latest , moonshot/random-choice:latest , moonshot/mew:latest ,

moonshot/memory:latest , moonshot/excel:latest , moonshot/date:latest ,

moonshot/base64:latest , moonshot/fetch:latest , moonshot/quickjs:latest ,

moonshot/code_runner:latest

Uses MOONSHOT_BASE_URL and MOONSHOT_API_KEY for OpenAI client

KIMI 开放平台



```
import os
import json
import asyncio

import argparse
import httpx
from openai import AsyncOpenAI

class FormulaChatClient:
    def __init__(self, moonshot_base_url: str, api_key: str):
        self.openai = AsyncOpenAI(base_url=moonshot_base_url, api_key=api_key)
        self.httpx = httpx.AsyncClient(
            base_url=moonshot_base_url,
            headers={"Authorization": f"Bearer {api_key}"},
            timeout=30.0,
        )
        self.model = "kimi-k2.6"

    async def get_tools(self, formula_uri: str):
        response = await self.httpx.get(f"/formulas/{formula_uri}/tools")
        return response.json().get("tools", [])

    async def call_tool(self, formula_uri: str, function: str, args: dict):
        response = await self.httpx.post(
            f"/formulas/{formula_uri}/fibers",
            json={"name": function, "arguments": json.dumps(args)},
        )
        fiber = response.json()

        if fiber.get("status", "") == "succeeded":
            return fiber["context"].get("output") or fiber["context"].get(
                "encrypted_output"
            )

        if "error" in fiber:
            return f"Error: {fiber['error']}"

        if "error" in fiber.get("context", {}):
```

KIMI

开放平台

```
        return f"Error: {fiber['context']['output']}"
    return "Error: Unknown error"
```



```
async def handle_response(self, response, messages, all_tools, tool_to_uri):
    message = response.choices[0].message
    messages.append(message)
    if not message.tool_calls:
        print(f"\nAI Response: {message.content}")
        return

    print(f"\nAI decided to use {len(message.tool_calls)} tool(s):")

    for call in message.tool_calls:
        func_name = call.function.name
        args = json.loads(call.function.arguments)

        print(f"\nCalling tool: {func_name}")
        print(f"Arguments: {json.dumps(args, ensure_ascii=False, indent=2)}")

        uri = tool_to_uri.get(func_name)
        if not uri:
            raise ValueError(f"No URI found for tool {func_name}")

        result = await self.call_tool(uri, func_name, args)
        if len(result) > 100:
            print(f"Tool result: {result[:100]}...") # limit the output length
        else:
            print(f"Tool result: {result}")

        messages.append(
            {"role": "tool", "tool_call_id": call.id, "content": result}
        )

    next_response = await self.openai.chat.completions.create(
        model=self.model, messages=messages, tools=all_tools
    )
    await self.handle_response(next_response, messages, all_tools, tool_to_uri)
```

KIMI

开放平台



```
response = await self.openai.chat.completions.create(
    model=self.model, messages=messages, tools=all_tools
)
await self.handle_response(response, messages, all_tools, tool_to_uri)

async def close(self):
    await self.httpx.aclose()

def normalize_formula_uri(uri: str) -> str:
    """Normalize formula URI with default namespace and tag"""
    if "/" not in uri:
        uri = f"moonshot/{uri}"
    if ":" not in uri:
        uri = f"{uri}:latest"
    return uri

async def main():
    parser = argparse.ArgumentParser(description="Chat with formula tools")
    parser.add_argument(
        "--formula",
        action="append",
        default=["moonshot/web-search:latest"],
        help="Formula URIs",
    )
    parser.add_argument("--question", help="Question to ask")

    args = parser.parse_args()

    # Process and deduplicate formula URIs
    raw_formulas = args.formula or ["moonshot/web-search:latest"]
    normalized_formulas = [normalize_formula_uri(uri) for uri in raw_formulas]
    unique_formulas = list(
        dict.fromkeys(normalized_formulas)
    ) # Preserve order while deduping

    print(f"Initialized formulas: {unique_formulas}")
```

KIMI

开放平台



```
api_key = os.getenv("MOONSHOT_API_KEY")
```

```
if not api_key:
```

```
    print("MOONSHOT_API_KEY required")
```

```
    return
```

```
client = FormulaChatClient(moonshot_base_url, api_key)
```

```
# Load and validate tools
```

```
print("\nLoading tools from all formulas...")
```

```
all_tools = []
```

```
function_names = set()
```

```
tool_to_uri = {} # inverted index to the tool name
```

```
for uri in unique_formulas:
```

```
    tools = await client.get_tools(uri)
```

```
    print(f"\nTools from {uri}:")
```

```
    for tool in tools:
```

```
        func = tool.get("function", None)
```

```
        if not func:
```

```
            print(f"Skipping tool using type: {tool.get('type', 'unknown')}")
```

```
            continue
```

```
        func_name = func.get("name")
```

```
        assert func_name, f"Tool missing name: {tool}"
```

```
        assert (
```

```
            func_name not in tool_to_uri
```

```
        ), f"ERROR: Tool '{func_name}' conflicts between {tool_to_uri.get(func_name, 'unknown')}"
```

```
        if func_name in function_names:
```

```
            print(
```

```
                f"ERROR: Duplicate function name '{func_name}' found across formulas")
```

```
            )
```

```
            print(f"Function {func_name} already exists in another formula")
```

```
            await client.close()
```

```
            return
```

KIMI

开放平台

```
tool_to_uri[func_name] = uri
print(f" - {func_name}: {func.get('description', 'N/A')}")
```



```
print(f"\nTotal unique tools loaded: {len(all_tools)}")
if not all_tools:
    print("Warning: No tools found in any formula")
    return

try:
    messages = [
        {
            "role": "system",
            "content": "你是 Kimi, 由 Moonshot AI 提供的人工智能助手, 你更擅长中文和英文"
        }
    ]
    if args.question:
        print(f"\nUser: {args.question}")
        await client.chat(args.question, messages, all_tools, tool_to_uri)
    else:
        print("Chat mode (type 'q' to quit)")
        while True:
            question = input("\nQ: ").strip()
            if question.lower() == "q":
                break
            if question:
                await client.chat(question, messages, all_tools, tool_to_uri)

finally:
    await client.close()

if __name__ == "__main__":
    asyncio.run(main())
```

相关概念和接口说明

理解 Kimi 官方工具之前，需要学习一个概念 'Formula'。Formula 是一个轻量脚本引擎集合。它可以将 Python 脚本转化为“可被 AI 一键触发的瞬态算力”，让开发者只需专注于代码编写，其余的启动、调度、隔离、计费、回收等工作都由平台负责。

Formula 通过语义化的 URI（如 moonshot/web-search:latest）来调用，每个 formula 包含声明（告诉 AI 能干什么）和实现（Python 代码），平台会自动处理所有底层细节（启动、隔离、回收等），让工具可以在社区中轻松分享和复用。您可以在 Kimi Playground 中体验和调试这些工具，也可以通过 API 在应用中调用它们。

调用官方工具的方法

对 formula uri，一般它由 3 个部分组成，比如 `moonshot/web-search:latest`。其中 web-search 部分是它的 `name`，namespace 目前我们只支持 `moonshot`，`latest` 会是默认的 tag。

一个典型的用法是如果我们需要调用 web search，可以发一个这样的 http request:

```
export FORMULA_URI="moonshot/web-search:latest"
export MOONSHOT_BASE_URL="https://api.moonshot.cn/v1"

curl -X POST ${MOONSHOT_BASE_URL}/formulas/${FORMULA_URI}/fibers \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $MOONSHOT_API_KEY" \
-d '{
  "name": "web_search",
  "arguments": "{\"query\": \"月之暗面最近有什么消息\"}"
}'
```

对 web-search，由于创建的时候设置为了 protected，它的结果会在

`context.encrypted_output` 字段出现。格式类似 `----MOONSHOT ENCRYPTED BEGIN-----` `--MOONSHOT ENCRYPTED END----`，这个内容可以塞到 tool 里面直接调用。

和 Chat Completions 的交互说明

Formula API 和模型对齐。

KIMI 开放平台
tools 字段怎么设置?



现在给定 formula uri 比如 `moonshot/web-search:latest`，我们可以直接把它拼接到 url 里面

```
curl ${MOONSHOT_BASE_URL}/formulas/${FORMULA_URI}/tools \  
-H "Authorization: Bearer $MOONSHOT_API_KEY"
```

一个样例输出是这样的:

```
{  
  "object": "list",  
  "tools": [  
    {  
      "type": "function",  
      "function": {  
        "name": "web_search",  
        "description": "Search the web for information",  
        "parameters": {  
          "type": "object",  
          "properties": {  
            "query": {  
              "description": "What to search for",  
              "type": "string"  
            }  
          },  
          "required": [ "query" ]  
        }  
      }  
    }  
  ]  
}
```

我们可以简单取 tools 字段 (总是一个 array of dict) 追加到你请求的 tools 列表中。我们总是保证这个 list 是 API 兼容的。

一个 API 的请求中是唯一的，不然这个 chat completion request 会被视为非法请求而立即被 401 返

KIMI 开放平台



此外，如果你同时使用了多个 formula，你需要自己维护一个 `function.name` → `formula_uri` 的这个映射，以备后用。

模型请求返回的处理

如果这个 chat completion 的返回 `finish_reason=tool_calls`，说明模型认为触发了工具调用的中断。这时候它内容可能类似是这样的：

```
{
  "id": "chatcmpl-1234567890",
  "object": "chat.completion",
  "choices": [
    {
      "message": {
        "role": "assistant",
        "tool_calls": [
          {
            "id": "web_search:0",
            "type": "function",
            "function": {
              "name": "web_search",
              "arguments": "{\"query\": \"天蓝色的 RGB 是什么?\"}"
            }
          }
        ]
      },
      "finish_reason": "tool_calls"
    }
  ]
}
```

我们通过 `choices[0].message.tool_calls[0].function.name` 发现需要调用 `web_search`，然后发现 `web_search` 对应的 `formula_uri` 是 `moonshot/web-search:latest`。

`${MOONSHOT_BASE_URL}/formulas/${FORMULA_URI}/fibers` 发出请求。特别的，因为模型输出

KIMI 开放平台 `arguments` 虽然内容是一个合法的 json，但是在格式上仍然是一个 encoded string。你不需要转义，直接作为调用的 body 就可以了。

Fiber 请求返回的处理

Fiber 是一次具体执行的“进程快照”，含日志、Tracing、资源用量，方便调试与审计。

POST 的结果一般是 `status` 可能是 `succeeded` 或者各种类型的错误，当 `succeeded` 后，结果可能类似如下：

```
{
  "id": "fiber-f43p7sby7ny111houyq1",
  "object": "fiber",
  "created_at": 1753440997,
  "lambda_id": "lambda-f3w8y6qcoqgi11h8q7ui",
  "status": "succeeded",
  "context": {
    "input": "{\"name\":\"web_search\",\"arguments\":{\"query\":\"天蓝色的\",
    \"encrypted_output\": \"----MOONSHOT ENCRYPTED BEGIN----+nf6...DSM=----MOONSHOT F
  },
  "formula": "moonshot/web-search:latest",
  "organization_id": "staff",
  "project_id": "proj-88a5894a985646b5902b70909748ba16"
}
```

特别的，如果是搜索，可能会返回的是 `encrypted_output`，而一般情况下我们可能返回 `output`。这个 output 就是你的下一轮输入。

一般继续请求的时候 messages 排列如下：

```
{
  "messages": [
    { /* 上一轮模型的返回内容 */
      "role": "assistant",
      "tool_calls": [
        {
          "id": "web_search:0",
          "type": "function",
          "function": {
            "name": "web_search",
            "arguments": "{\"query\": \"天蓝色的 RGB 是什么?\"}"
          }
        }
      ]
    },
    { /* 你需要补充的信息 */
      "role": "tool",
      "tool_call_id": "web_search:0", /* 注意这儿的 id 需要和前面的 tool_calls[].id 对齐 */
      "content": "----MOONSHOT ENCRYPTED BEGIN----+nf6...DSM----MOONSHOT ENCRYPTED"
    }
  ]
}
```

接下来模型就可以做进一步的推理了。

注意要点：

- 模型可能会返回超过一个 tool_calls，因此你必须对所有 tool_calls 都给出返回模型才会继续，否则会认为请求不合法而拒绝请求
- assistant 如果带 tool_calls，接下来必定是和 tool_calls 完全一致的几个 role=tool 的 message，并且 tool_call_id 要求和前面的 tool_calls.id 一一对齐。
 - 如果有多个 tool_calls 顺序不敏感
 - 我们模型输出的 tool_calls 的几个 id 一定是唯一的，后面 role=tool 时候 id 也必须对齐
 - 仅在当轮这个 tool_calls - response 的局部有唯一性要求，对整个 conversation 或者全局这个唯一性不敏感

 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

KIMI 开放平台



< 联网搜索工具

ModelScope MCP 配置 >

